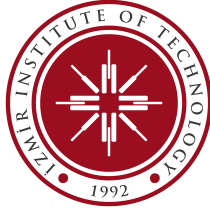


CENG318 Human Computer Interaction Spring 2026

**Project Title: BarberNearMe
Group 19 Final Report**

June 21, 2026



Group Members

- Vedat Demir 280201047
- Osman Said Tekin 270201010
- Burak Rahmanoğlu 290201030
- Batuhan Mutlu 300201094
- Enes Ergün Hoşgör 300201053

Abstract

BarberNearMe is a cross-platform mobile application that connects customers with nearby barbershops and digitises an appointment lifecycle still dominated by walk-ins and phone calls. This report documents the move from proposal to a working prototype, organised around the three pillars introduced earlier: Discovery, Interaction, and Reliability. For Discovery, the app combines device GPS, an interactive map, and client-side Haversine distance computation to rank shops by proximity, rating, price, and open-now status. For Interaction, customers browse shop details, book conflict-free time slots, manage and cancel appointments, exchange real-time messages with shops, and rate completed visits; barbers register a shop through a multi-step wizard and manage appointments from a dashboard. For Reliability, reviews are gated so that only the customer of a *completed* appointment may post exactly one review per visit, structurally enforcing the verified-only rating goal. During development the stack changed substantially: instead of an Android/Kotlin client with a .NET 8 REST API over PostgreSQL, the team built an Expo / React Native (TypeScript) client backed by Firebase Authentication and Cloud Firestore, accessed through a dedicated service layer. The payment step uses a mock deposit flow secured by Luhn check-digit validation, verified against a 100-card test set (50 valid, 50 invalid) with 100% classification accuracy; one proposal item remains open: the AI head-shape styling module is future work. Formal usability evaluation (System Usability Scale) and performance benchmarking were planned but not yet executed; their methodology is specified here and their results are explicitly marked as pending. The outcome is a functional, end-to-end booking prototype with a clearly scoped path to completion.

1 Introduction

Personal-care services such as barbershops still rely heavily on manual booking and word-of-mouth reputation. *BarberNearMe* targets this gap with a single mobile application where customers discover well-rated local barbers, book a slot, communicate with the shop, and leave trustworthy feedback. This document is a progress report: rather than restating intentions, it records what was built, what diverged from the proposal, and what remains.

The project addresses three concrete problems that motivated the design:

- **Service uncertainty and wait times.** Customers cannot easily compare nearby shops or see availability before showing up. *BarberNearMe* answers this with map- and list-based discovery, proximity ranking, an open-now filter, and a calendar that disables already-booked slots to prevent double-booking.
- **The trust deficit in reviews.** Most platforms let anyone post a rating. *BarberNearMe* ties every review to a specific completed appointment owned by the reviewer and permits exactly one review per appointment, so feedback comes only from people who actually received the service.
- **The styling mismatch.** A chosen hairstyle may not suit a customer's features. This is the most forward-looking problem: the architecture leaves room for an image-based head-shape analysis module, which is documented here as future work rather than a delivered feature.

The remainder of the report reviews related work and positions the novelty of the approach (Section 2), describes what was actually implemented across the six planned stages (Section 3), states the evaluation plan and the current evidence (Section 4), and concludes with an honest account of remaining work (Section 5).

2 Literature Review

Commercial scheduling platforms such as Booker and StyleSeat focus mainly on administrative booking and calendar management. They are effective for shop owners but offer limited support for the two problems most relevant here: the authenticity of reviews and the match between a hairstyle and a customer's physical features. In particular, these platforms generally do not verify that a reviewer actually received the service, which makes ratings vulnerable to bias, and they provide no visual guidance for style compatibility.

From a human-computer interaction standpoint, the design of *BarberNearMe* follows established usability principles—consistency, user control, recognition over recall, and clear feedback—as articulated by Shneiderman et al. (2016). These principles are reflected concretely in the prototype: a persistent bottom-tab structure, status badges on appointments, disabled time slots that communicate availability at a glance, and confirmation dialogs before destructive actions such as cancellation.

The novelty of *BarberNearMe* relative to existing tools is the combination of (i) a *verified-only* review model in which feedback is bound to a completed transaction, (ii) a deposit-based booking commitment secured by Luhn check-digit validation (Luhn, 1960), and (iii) a planned vision-based styling aid. The styling aid would build on classic real-time face/feature detection such as the classic boosted-cascade detector of Viola and Jones (2001), which remains a practical baseline for on-device facial-region detection. Of these three, the verified-review model and the Luhn-secured deposit step are fully implemented and tested, while the vision module is not yet started; the corresponding citations are therefore used to ground both the delivered design and the future-work direction.

3 Methodology

Development followed the six iterative stages set out in the proposal. The most important change to report up front is the technology stack. The proposal planned an Android/Kotlin client, a .NET 8 REST API, and a PostgreSQL database. In practice the team built a single cross-platform client with Expo / React Native and TypeScript, and used Firebase Authentication together with Cloud Firestore as the backend. The switch was a deliberate scope decision rather than an oversight: a five-person team with a fixed deadline could not maintain a native Android codebase, a separate .NET server, and a managed database in parallel, so a backend-as-a-service was adopted to maximise feature velocity at the cost of some originally planned server-side guarantees (transactional double-booking checks in particular), tracked explicitly as future work in Section 5. All UI code reaches data exclusively through a dedicated service layer (`src/services/`), so the application keeps a clean separation between presentation and data access even though it uses a backend-as-a-service rather than a hand-written REST server. The service-layer boundary preserves the resource-oriented, stateless client-server discipline of the REST architectural style (Fielding, 2000), which is the principle the original .NET API was meant to embody.

Table 1: Firestore data model (as implemented).

Collection	Key fields
users	firstName, lastName, email, phone, role (customer / barber)
barbers	shopName, services[], staff[], workingHours, location (GeoPoint), rating, reviewCount, isActive, walletBalance
appointments	customerId, barberId, staffId, serviceId, serviceName, servicePrice, staffName, totalPrice, durationMin, date, timeSlot, status, kaporaPaid, kaporaAmount
reviews	customerId, barberId, appointmentId, customerName, rating, qualityRating, cleanlinessRating, timelinessRating, comment
conversations messages	/ deterministic customer_barber id; messages with text, senderRole, timestamp

3.1 Stage 1 – Requirement Analysis and Personas

Functional requirements were derived for two roles, Customer and Barber, and are realised directly in the navigation graph: an unauthenticated stack (intro, login, sign-up, barber registration) and two role-specific authenticated stacks selected at runtime from the signed-in user’s `role` field. The driving persona remains a student in the Urla/Gülbahçe area near IZTECH looking for a nearby, well-rated barber—reflected in the seeded data, which uses real shops and coordinates in that region.

3.2 Stage 2 – System Design and Data Modeling

The relational PostgreSQL schema from the proposal was replaced by a document model in Cloud Firestore. Five collections capture the domain, summarised in Table 1; conversations and messages are stored as two linked collections but are shown as a single entity in the diagram for clarity. To keep the prototype within Firebase’s free tier and avoid composite-index setup, list queries fetch by a single equality filter and perform sorting and secondary filtering on the client. Figure 1 shows the resulting entity-relationship diagram: the collections are connected by reference fields (e.g. `customerId, barberId`) rather than enforced foreign keys, since Firestore does not support relational constraints. The schema also lives in the TypeScript interfaces and the project README.

3.3 Stage 3 – Frontend (Mobile UI)

The client was implemented with React Native instead of Kotlin. The Customer side is complete end to end: a Home screen with a `react-native-maps` map, search, a preferences sheet (sort by recommended / nearest / top-rated / best-price, service-type chips, open-now switch, and a draggable distance-radius slider), a shop-detail screen, a slot-based booking screen, an appointments list with upcoming/past/cancelled tabs and status badges, a messaging screen, a rating screen, and a profile. The rating screen captures three separate dimensions—service quality, cleanliness, and timeliness—in addition to an overall score and a free-text comment, giving more actionable feedback than a single star rating. The Barber side provides a four-step registration wizard, a dashboard, an appointments view, and a shop profile. Discovery distance is computed on the client with the Haversine formula against the device’s GPS position obtained via `expo-location`.

3.4 Stage 4 – Backend Integration and Logic

Authentication uses Firebase email/password; profiles and barber-shop placeholders are created on sign-up. The booking flow writes an appointment document, and double-booking is prevented by reading the day’s existing `pending/confirmed` slots and disabling them in the UI, taking service duration into account. Appointment status transitions (`pending`, `confirmed`, `completed`, `cancelled`) are persisted, and cancellation is allowed only up to two hours before the slot.

The payment step is a deliberately scoped mock, but it is no longer a bare field-completeness check. The implemented `PaymentScreen` validates the card number with the Luhn check-digit algorithm (Luhn, 1960) (`isValidLuhn` in `src/utlis/luhn.ts`), in addition to checking that the expiry, CVV, and cardholder fields are filled; only a Luhn-valid number unlocks the “pay deposit” action. The function was verified with a dedicated test script against 6 known card numbers (industry-standard test numbers such as 4242 4242 4242 4242, plus deliberately corrupted variants) and against a generated 100-card set (50 valid, 50 invalid), achieving 100% classification accuracy on both – the exact test specified in the proposal’s Experimental Results plan. On a successful charge the flow transfers a 10% deposit (“kapora”) to the barber’s wallet balance and creates the appointment. No real payment gateway is involved; this remains a simulated transaction by design.

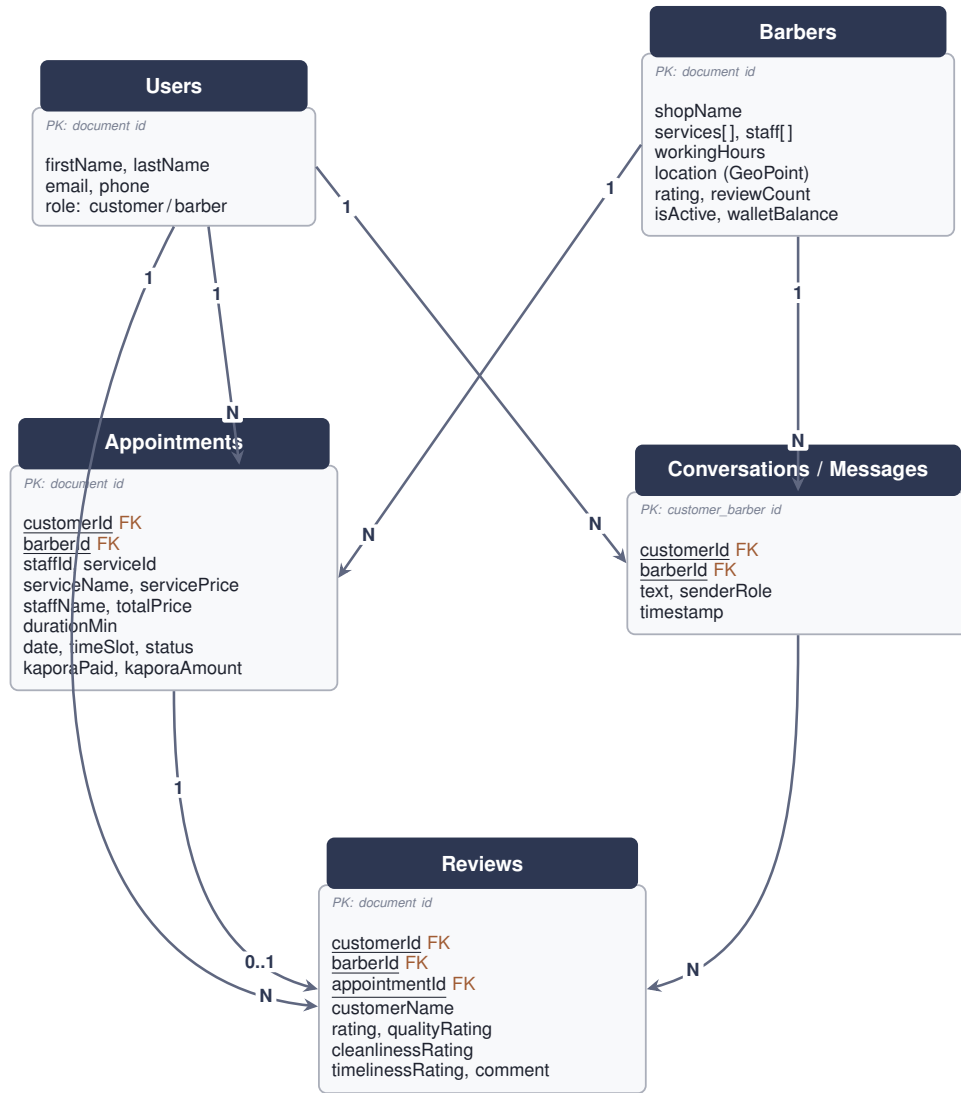


Figure 1: Entity-relationship diagram of the implemented Firestore data model. Underlined fields are reference (FK-style) fields; Firestore does not enforce them as true foreign keys. Cardinalities reflect the application logic (e.g. one appointment maps to at most one review, per the verified-only review rule).

3.5 Stage 5 – AI Module Research (Future Integration)

No computer-vision code exists in the repository. The intended direction is documented rather than built: a head-shape/face-region detector based on the Viola–Jones cascade (Viola and Jones, 2001) (or a modern equivalent) feeding a rule-based style recommender. This stage is explicitly future work.

3.6 Stage 6 – Usability Testing and Quality Assurance

The planned evaluation method is inspection- and questionnaire-based usability assessment in the tradition of Nielsen’s usability inspection methods (Nielsen, 1994), with the System Usability Scale (SUS) as the summative instrument. Aside from the Luhn unit test in `src/utlis/luhn.test.ts` (Section 4), the repository contains no automated UI tests, no continuous-integration configuration, and no recorded usability-study data, so the outcomes of this stage are reported as pending in Section 4.

4 Experimental Results

To respect academic integrity, this section separates what has been *verified* from what is only *planned*. At the time of writing the repository contains no automated UI test suite, CI pipeline, or survey data, so usability and runtime-performance numbers cannot be reported yet. One quantitative result does exist – the Luhn payment check has been unit-tested and is reported with real numbers below. The measurement plan for the remaining targets specifies how each will be evaluated, with concrete TODOs for the missing numbers.

4.1 Functional verification (evidenced)

The following behaviours are verifiable from the implemented code and a running build:

- Discovery filters and sorts (nearest, top-rated, best-price, open-now, distance radius) operate over both seeded Firestore data and a built-in mock fallback.
- Booking disables already-taken slots for the selected day and barber, preventing double-booking in the UI.
- The verified-review rule holds: a “Rate” action appears only on completed appointments, and a second review for the same appointment is blocked.
- Messaging updates in real time via Firestore `onSnapshot` subscriptions.
- **Payment validation (Luhn).** `isValidLuhn` was tested against 6 known card numbers (industry-standard test numbers and deliberately corrupted variants) and against a generated 100-card set (50 valid, 50 invalid), via `src/utlis/luhn.test.ts`. Result: 6/6 known-card checks passed and 100/100 (100%) classification accuracy on the generated set – the exact test specified in the proposal.

4.2 Usability and HCI metrics (planned)

- **System Usability Scale (SUS).** Target ≥ 75 (“high usability”). Method: standard 10-item SUS administered after a guided task set.
- **Task-completion rate.** Percentage of users who complete a booking without assistance or error.
- **Booking efficiency.** Average in-app booking time vs. a phone/walk-in baseline.

4.3 Remaining technical validation (planned)

- **Discovery latency.** The proposal’s “<200 ms geospatial query” target should be reframed for the actual architecture: distance is computed on the client and shop data is fetched with a single-filter Firestore query. The meaningful metric is barber-list load time (network fetch + client sort).
- **Booking integrity.** Concurrency behaviour under simultaneous bookings of the same slot.

5 Conclusions and Future Works

Discovery is delivered: GPS, map, and client-side distance ranking with practical filters work over real seeded shops near IZTECH. **Interaction** is delivered end to end for customers (booking, cancellation, real-time messaging, rating) and substantially for barbers (registration wizard, dashboard, appointment management). **Reliability** is substantially delivered: the verified-only review rule is enforced, and the deposit step is secured by a unit-tested Luhn check (100% accuracy on a 100-card set), though it remains a simulated transaction with no real payment gateway by design. The largest deviation from the proposal is the stack itself—Expo/React Native and Firebase replaced Kotlin, .NET, and PostgreSQL—which let a small team ship a cross-platform, end-to-end prototype quickly at the cost of a custom server and some originally promised server-side guarantees.

Even at prototype scale the work has value. It demonstrates a concrete, testable mechanism for the recurring trust problem in service-review platforms (binding feedback to a completed transaction), and it packages location discovery, slot-based booking, and direct messaging into one coherent HCI flow grounded in established usability principles (Shneiderman et al., 2016).

Future works, in priority order: (1) move the double-booking guard into a Firestore transaction/security rule; (2) run the SUS-based usability evaluation (Nielsen, 1994) and the performance measurements specified in Section 4; (3) finalise remaining barber-side polish and add automated tests and CI; and (4) build the AI head-shape styling module (Viola and Jones, 2001), the proposal’s explicitly forward-looking feature.

6 Weekly Schedule / Project Plan

Table 2 recolours the proposal’s 12-week plan to reflect the real state of the repository: **finished**, **in progress**, and **planned / future**. Task labels are updated where the implementation diverged from the plan (e.g. React Native instead of Kotlin, Firebase instead of .NET/PostgreSQL). Note that the actual coding activity was concentrated in a short, intense sprint rather than spread evenly across twelve calendar weeks; the columns therefore represent plan phases, while the colours represent delivery status as of this report.

Table 2: Project plan, recoloured by actual delivery status. Green = finished, blue = in progress, yellow = planned/future.

Task	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12
Requirement analysis & personas	finished	finished										
UI/UX wireframing (low-fi)		finished	finished									
Data modeling (Firestore) & ERD			finished	finished								
Backend / data layer (Firebase; was .NET)				finished	finished	finished	finished					
Mobile UI (React Native; was Kotlin)					finished	finished	finished	finished				
Real-time messaging							finished	finished	finished			
Payment (mock deposit, Luhn-secured)							finished	finished	finished			
AI infrastructure research (head-shape)								planned / future	planned / future	planned / future		
System integration & debugging									finished	finished	finished	
Usability testing (SUS) & final fixes											planned / future	planned / future
Final demo & presentation												planned / future

References

- Fielding, R. T. (2000). *Architectural Styles and the Design of Network-Based Software Architectures*. Ph. D. thesis, University of California, Irvine.
- Luhn, H. P. (1960). Computer for verifying numbers. U.S. Patent 2,950,048. Filed 23 Jan. 1954, granted 23 Aug. 1960.
- Nielsen, J. (1994). Usability inspection methods. *Conference Companion on Human Factors in Computing Systems (CHI)*, 413–414.
- Shneiderman, B., C. Plaisant, M. Cohen, S. Jacobs, N. Elmqvist, and N. Diakopoulos (2016). *Designing the User Interface: Strategies for Effective Human-Computer Interaction* (6th ed.). Pearson.
- Viola, P. and M. Jones (2001). Rapid object detection using a boosted cascade of simple features. In *Proc. IEEE Computer Society Conf. on Computer Vision and Pattern Recognition (CVPR)*, Volume 1, pp. I–511–I–518. IEEE.